

Online Performance Visualization of Parallel Applications

Sagnik Dey*

Purdue University

*Work done while at IIT Kanpur

Preeti Malakar

Indian Institute of Technology Kanpur

Abstract—Monitoring the performance of a large-scale long-running parallel application is challenging. Most tools capture the entire execution trace and perform post hoc analysis and consumes gigabytes of memory. Our work aims to visually analyze communication performance on-the-fly while the program is running.

I. INTRODUCTION

Motivation: Large-scale parallel applications routinely run on hundreds of compute nodes in supercomputers/clusters. One of the main challenges in parallel computing is achieving good scalability of applications. This may be due to communication bottlenecks, the network topology, the computation-to-communication ratio and code inefficiency, among other factors. Application developers often look at the communication time profiles to investigate the performance bottlenecks. A detailed visual analysis of communication times for different communication functions across all processes at regular intervals along with the running application may provide helpful insights to the application developer.

Limitation of the state-of-the-art approaches: Several parallel profilers such as mpiP, Vampir, TAU and HPCToolkit [1], [2] are routinely used by MPI (message passing interface) application developers to find potential scalability bottlenecks. However, these profilers dump the profiles at the end of the execution. This gives an overall view of the computation and communications. Profiling may not reveal intermediate communication events in detail. In contrast, tracing helps to perform a timestamped in-depth analysis of the entire application execution. However, trace files may be in the range of gigabytes which makes the trace analysis process challenging and time-consuming [3]. Thus, a feasible solution is to view intermediate profiles of applications on-the-fly without incurring high trace analysis times and trace storage requirements. Chimbuko [4] is a tool that does online anomaly detection of MPI performance. However, our goal is a generic online (communication) performance profiler.

Key insights and contributions: We have developed a lightweight visualization tool that visualizes and analyzes application performance on-the-fly using the mpiP profiler [1], a light-weight MPI profiler. This helps users easily view intermediate performance (using mpiP profiles) from a running MPI application. We have extended the functionality of mpiP to generate profiles periodically in order to visualize intermediate performance metrics. Our tool captures and visualizes performance metrics exposed by mpiP [1] such as time spent in MPI functions, aggregate message sizes and rank-wise time for large scale MPI applications. Our tool accumulates these statistics periodically (at an user-set frequency) and merges the per-rank statistics into mpiP profiles. These profiles are sent to the user's visualization and analysis

These profiles are parsed to visualize the real-time performance of the running application on the visualization system (local machine). We refer to this as online (on-the-fly/real-time) performance visualization.

II. ONLINE PROFILER

Large-scale parallel applications based on message passing interface programming paradigm require high compute power of supercomputers. We have developed a tool based on the popular mpiP profiler [1] to perform online visual analysis of MPI programs when they are in execution. In this section, we describe the internals of this tool. We extended the mpiP library to enable generation of intermittent .mpiP profile files. The application to be profiled is compiled with our modified mpiP library (shown in the top left rectangle of Figure 1). The mpiP profiles are generated at an user-specified interval (set to 60 seconds by default). These are written to the shared storage of cluster, this may be any form of storage as configured/desired by the user. We transfer these profiles as soon as they are generated to the user's local system for visualization as described in §III.

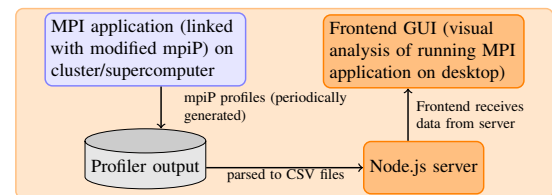


Fig. 1: Real-time visualization of intermittent performance.

We now describe the modifications required in mpiP source code. mpiP captures performance metrics of MPI programs via instrumentation of all MPI functions using the PMPI profiling layer. We extended mpiP to dump the profiles periodically so that we can monitor the detailed time-varying performance of MPI applications. We modified mpiP to use multi-threaded mode via MPI_Init_thread to use threads along with MPI processes. At the mpiP initialization phase, a new thread is launched for each MPI process. The threads are created using the Pthreads library and every thread functions as profile generator thread in each rank. The thread collects intermittent profile data from the memory of each process at a user-specified interval. All threads coordinate and collate the performance statistics and the aggregated profiles are output to .mpiP profile files periodically. The main MPI application continues execution; MPI ranks do not wait for data collection. The threads collect data and write the profiles to .mpiP files, the main MPI rank does not need to wait. The .mpiP files are time-stamped to identify the different snapshots generated during the lifetime of the application.

III. ONLINE PERFORMANCE VISUALIZATION

Our goal is to provide fast, easy-to-comprehend, time-varying and real-time view and analysis of MPI program performance. We developed a real-time web-based visualization of the ongoing MPI application performance data using the popular D3.js library. This enables real-time performance visualization from any system using the URL of the local system. The `mpiP` profiles are transferred (via *sftp*) from the cluster/supercomputer to the user's local machine periodically. An analysis engine (a Python application running in the background) parses the `.mpiP` profiles and generates `.csv` files as soon as a new profile output is received. A Node.js server runs in the background and reads these `.csv` files and displays on the front-end according to the selected visualization.

The visualization front-end is written using D3, HTML, CSS and JS. We have designed several different visualizations ranging from bar charts to heatmaps with process-specific views and function-specific views. This may be used to view various performance statistics of the running application to diagnose performance bottlenecks. This will help the programmer to find specific processes and functions that may be potential bottlenecks. Visual analysis is helpful especially in case of performance analysis of a parallel program running on thousands of nodes. We show snapshots periodically (60 seconds by default) in our GUI (automatically updated). The user may navigate back and forth in time and also keep track of interesting snapshots. Our tool supports low bandwidth remote visualization because the individual profiles that are transferred over the network are less than 1 MB per snapshot.

IV. EXPERIMENTS AND RESULTS

We visualize real-time performance data related to MPI Time, Message Sent Statistics, Time Statistics, P2P Time, Collective Time as reported by `mpiP`. These help in detailed analysis of time spent by each rank in MPI functions, message size per rank, mean time taken by each function, percentage of time spent in point-to-point and collective MPI communications during every snapshot profiled during the execution. The launch page of our visualization front-end provides several navigation options. The default is pie-chart view of communication time consumed by various MPI functions in the last six snapshots. We used bar charts, heat maps and pie charts in D3. We now show a few visualizations obtained while running the classical molecular dynamics simulation, LAMMPS, with 51 million atoms on up to 32 processes (2 Intel Xeon Gold nodes). It took only 21 seconds to transfer each `.mpiP` profile from the mini-cluster to our local machine for visualization.

Figure 2 shows the statistics of messages sent during two successive snapshots (interval of 60s). The Y-axis is the mean time taken by a single instance of the function during that interval and the color is determined by the total time taken by all instances of the function. This is equal to the `count` times the mean time. `count` is the number of invocations of the function during that interval. These metrics are reported in the `.mpiP` profile file. The X-axis shows the MPI function names. The functions appearing in the X-axis in a snapshot indic

the functions invoked during that interval. These highlight the variation in the time taken for a given MPI function and the variation of functions invoked during the application execution.

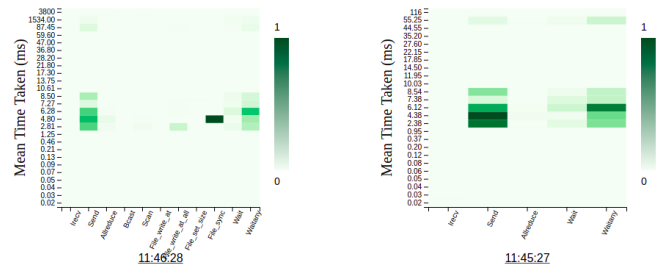


Fig. 2: Visualization of average time taken per MPI function during two successive snapshots of LAMMPS simulation on 32 processes.

Figure 3 shows the percentage of time spent in each MPI collective function per data size bin for LAMMPS run on 8 and 32 processes during the same snapshot. Y-axis shows the percentage of time spent in each function. X-axis shows the MPI function and data size bin information. The data size bin for the function calls are shown in parenthesis above each bar along with the function name. Each MPI function is shown in a different color. Note that MPI_Bcast calls (shown in sky blue) dominate the communication times. MPI_Bcast calls for 64K-128K message size took more than 60% on 8 processes and less than 30% on 32 processes. This may be due to more parallel message transfers and strong scaling. Time-varying view of aggregate statistics may help users examine time-varying performance of the collectives.

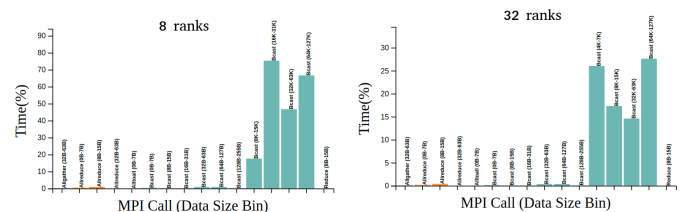


Fig. 3: Aggregate collective times for the same intermediate snapshot of LAMMPS simulation on two different process counts.

V. CONCLUSIONS

Our online visual profiler visualizes the intermittent mpiP profiles generated at the simulation site at the user’s local system. We use threads for real-time data collection and storage/transfer, thus the main MPI application is not stalled. We visualize various communication statistics on-the-fly depicting a real-time performance view of the application.

REFERENCES

- [1] J. Vetter and C.ambreau, “mpiP: Lightweight, scalable MPI profiling,” *URL: <https://software.llnl.gov/mpiP/>*, 01 2005.
- [2] S. S. Shende and A. D. Malony, “The Tau Parallel Performance System,” *The International Journal of High Performance Computing Applications*, vol. 20, no. 2, pp. 287–311, May 2006.
- [3] H. Mickler and et al., “Trace-Based Analysis and Optimization for the Semtex CFD Application – Hidden Remote Memory Accesses and I/O Performance,” in *Euro-Par Workshops*, 2009, pp. 295–304.
- [4] C. Kelly and et al., “Chimbuko: A Workflow-Level Scalable Performance Trace Analysis Tool,” in *ISAV’20 In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*, 2020.